Intune Administrative Governance — RBAC & Multi-Admin Approval

Implementation Solution Guide

Version 1.0.0 · 2026-06-30

Contents

Executive Summary — Intune Administrative Governance: RBAC & Multi-Admin Approval (Microsoft Intune) 4

What you get . . . 4

What this delivers . . . 4

The Phoenixtekk deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Intune Administrative Governance (RBAC & Multi-Admin Approval) 5

What Intune administrative governance is . . . 5

How it works . . . 5

What you can control . . . 5

Why this matters (the why lives here, in the Overview) . . . 6

Limitations (be honest with buyers) . . . 6

The standard Phoenixtekk deployment model . . . 6

2. Requirements Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval) 6

2.1 Intune licensing . . . 7

2.2 Entra ID groups for admins and approvers . . . 7

2.3 Operator holds an RBAC-capable role . . . 7

2.4 A break-glass administrator (do not lock yourself out) . . . 7

2.5 An admin-role design (least privilege) . . . 8

2.6 A scope-tag taxonomy . . . 8

2.7 A list of high-risk resource types to gate . . . 8

2.8 A change-control process for approvals . . . 8

2.9 Network access to Microsoft Graph / Intune endpoints . . . 8

Checklist . . . 8

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 9

G5. Pilot security group . . . 9

G6. Supported Windows baseline . . . 9

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval) 10

Step 1 — Build the admin and approver scope groups . . . 10

Step 2 — Create the scope tag . . . 10

Step 3 — Create the custom roles . . . 10
Step 4 — Assign the custom role . . . 11
Step 5 — Apply the scope tag to objects . . . 11
Step 6 — Enable multi-admin approval for high-risk apps . . . 11
Step 7 — Enable multi-admin approval for high-risk scripts . . . 11
Step 8 — Pilot one approval (the pilot gate) . . . 12
Step 9 — Validate and report . . . 12
Step 10 — Expand to production . . . 12
5. Testing Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval) 12
Test 1 — Custom roles exist and are scoped . . . 12
Test 2 — Role assignment is correct . . . 13
Test 3 — Scope tags limit visibility . . . 13
Test 4 — Least privilege holds . . . 13
Test 5 — Access policies exist and target the approver group . . . 13
Test 6 — A gated change requires approval . . . 13
Test 7 — Rejection blocks the change . . . 13
Test 8 — Governance coverage report . . . 13
Pass/fail gate . . . 13
6. Troubleshooting Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval) 14
A. Admin can see/do too much or too little . . . 14
B. Multi-admin approval not gating changes . . . 14
C. Locked out (no one can make changes) . . . 15
D. Pending approval requests stuck . . . 15
E. Scripts cannot read governance data . . . 15
F. Diagnostics quick reference . . . 15
G. When to escalate . . . 15
7. Best Practices — Intune Administrative Governance (RBAC & Multi-Admin Approval) 15
8. Deliverables Summary — Intune Administrative Governance (RBAC & Multi-Admin Approval) 16
Documentation . . . 16
PowerShell scripts . . . 17
What the buyer receives . . . 17
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — Intune Administrative Governance: RBAC & Multi-Admin Approval (Microsoft Intune)

Phoenixtekk Workplace Marketplace · Intune Solutions

When everyone is an Intune Administrator, mistakes and blast radius follow — a bad app assignment or a destructive script can hit the whole fleet, and a single compromised admin account can do unlimited damage. Modern governance fixes both with least-privilege RBAC and a second pair of eyes on risky changes. This solution implements Intune administrative governance: custom roles and scope tags that limit who can touch what, plus access policies (multi-admin approval) so high-risk actions — such as app or script deployments — require a second admin to approve before they go live.

This is a complete, deploy-it-today package for governing Intune administration, written for an administrator with no prior RBAC experience.

What you get

• A plain-language Solution Overview of Intune RBAC (built-in vs custom roles, permissions, scope tags, scope groups) and the multi-admin approval / access-policy model — and why both matter.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, including a break-glass admin so you never lock yourself out.

• An Implementation Guide of precise, ordered Intune steps: design roles and scope tags, create custom roles and assignments, tag objects, enable access policies for high-risk resource types, then pilot, validate, and expand.

• Five PowerShell scripts for governance discovery, role-assignment and approval-policy validation, a coverage and over-privilege report, safe remediation of over-privileged assignments, and an admin/approver scope-group builder.

• Testing and Troubleshooting guides for a confident rollout.

What this delivers

A least-privilege RBAC model — custom roles such as `App Manager – Region A – Prod` bounded by scope tags such as `Region A – Devices – Prod` — combined with multi-admin approval gates (`App Deploy Approval – All – Prod`, `Script Deploy Approval – All – Prod`) that require a second admin to approve high-risk changes, all proven on a pilot before broad enforce.

The Phoenixtekk deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

Intune locked down to least privilege, with a required second-admin approval on high-risk changes — so mistakes and blast radius shrink, separation of duties is enforced, and every risky change has an audit trail

— deployable by a single administrator and validated before fleet rollout.

1. Solution Overview — Intune Administrative Governance (RBAC & Multi-Admin Approval)

What Intune administrative governance is

Intune administrative governance is the practice of controlling *who* can administer Intune, *what* they can touch, and *which* high-risk changes need a second person to sign off. It combines two native Intune capabilities:

1. Role-based access control (RBAC) — built-in and custom roles, granular permissions, and scope tags that limit each admin to a defined slice of objects and a defined set of actions.
1. Multi-admin approval (access policies) — `operationApprovalPolicies` that require a second admin to approve high-risk operations (such as app or script deployments) before they take effect.

Together they replace the "everyone is an Intune Administrator" anti-pattern with least-privilege roles and a required second pair of eyes on the changes that can break a fleet.

How it works

1. An Intune custom role bundles a precise set of permissions (for example, manage apps but not manage devices) into a reusable definition under `roleDefinitions`.
1. A scope tag (under `roleScopeTags`) is a label applied to Intune objects (apps, policies, profiles) and to a role assignment, so an admin only sees and manages the objects that carry the same tag — a who-touches-what boundary.
1. A role assignment (under `roleAssignments`) binds a role definition to admin members, targets a set of managed objects (scope groups), and carries scope tags — defining the admin's effective reach.
1. An access policy (under `operationApprovalPolicies`, surfaced in the portal as Multi admin approval) names a resource type (for example, apps or scripts) and an approver group; any create/change/delete of that resource type is held as a request until a different admin in the approver group approves it.
1. Approvers review pending requests, approve or reject with a note, and only then does the change apply — so no single admin can unilaterally push a high-risk change.

What you can control

- Built-in vs custom roles — start from a built-in role or define a custom role with exactly the permissions an admin needs.
- Permissions — granular read/create/update/delete rights per Intune workload.

• Scope tags — partition objects (by site, business unit, platform) so admins manage only their slice.

• Scope groups — the admin members and the targeted managed groups on each assignment.

• Access policies (multi-admin approval) — which resource types require approval and which group approves them.

Why this matters (the why lives here, in the Overview)

• Blast radius — when every admin is a full Intune Administrator, one mistake (a bad app assignment, a destructive script) can hit the whole fleet. Least-privilege roles and scope tags shrink the radius of any single account.

• Insider and compromised-account risk — a stolen or rogue admin credential can do far less when it holds a narrow custom role, and high-risk actions still need a second approver.

• Separation of duties — multi-admin approval enforces that the person who *requests* a high-risk change is not the person who *approves* it — a control auditors expect.

• Accountability and audit — named roles, scope tags, and approval records make it clear who can do what and who approved each risky change.

Limitations (be honest with buyers)

• Multi-admin approval covers specific resource types (such as apps and scripts); it is not a blanket approval gate over every Intune action. Confirm the current supported resource types in the portal.

• Scope tags govern visibility/management of taggable Intune objects; not every object type supports tagging — verify coverage for the workloads you care about.

• RBAC controls Intune; tenant-wide identity roles live in Entra ID and are governed separately.

• Over-tightening can lock admins (or yourself) out — always keep a break-glass administrator outside the restricted roles and outside the approval gate.

The standard Phoenixtekk deployment model

> *Design roles and scope tags, build admin/approver scope groups, create custom roles and assignments, apply scope tags to objects, enable access policies for high-risk resource types, then pilot one approval before broad enforce. Phoenixtekk pattern: Assess → Configure → Pilot → Validate → Expand.*

2. Requirements Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval)

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Named artifacts used throughout this solution (create them in section 2.6–2.9 and reuse the names verbatim):

Artifact	Name
Custom role	`App Manager – Region A – Prod`
Custom role	`Policy Reader – All – Audit`
Scope tag	`Region A – Devices – Prod`
Admin scope group	`Intune Admins – Region A – Prod`
Approver scope group	`Change Approvers – Apps – Prod`
Role assignment	`App Manager Assignment – Region A – Prod`
Access policy	`App Deploy Approval – All – Prod`
Access policy	`Script Deploy Approval – All – Prod`

2.1 Intune licensing

Description: Intune (Plan 1 or via M365 E3/E5/Business Premium). RBAC and multi-admin approval are built-in Intune features. Why: Roles, scope tags, and access policies are delivered by Intune. Verify: M365 admin center → Billing → Licenses. Implement: Assign M365 E3/E5 or Business Premium (Intune included).

2.2 Entra ID groups for admins and approvers

Description: Microsoft Entra security groups for admin members and for approvers. Why: Role assignments target admin groups; access policies target an approver group. Verify: Entra → Groups; confirm the groups exist (or run `RBACGOV-Discovery.ps1`). Implement: Entra → Groups → New group → Security; or build them with `RBACGOV-Set-ScopeTargeting.ps1`. Use the names `Intune Admins – Region A – Prod` and `Change Approvers – Apps – Prod`.

2.3 Operator holds an RBAC-capable role

Description: The person configuring governance holds Intune Administrator (or Global Administrator for the initial setup). Why: Creating role definitions, scope tags, and access policies requires elevated rights. Verify: Entra → Roles and administrators → confirm Intune Administrator. Implement: Assign Intune Administrator to the operator; reserve Global Administrator for tenant-wide changes.

2.4 A break-glass administrator (do not lock yourself out)

Description: At least one emergency Global Administrator account that is excluded from the restricted custom roles and from the multi-admin approval gate. Why: Tight RBAC plus an approval gate can lock out all working admins; a break-glass account is the recovery path. Verify: Entra → Roles and administrators → Global Administrator → confirm a dedicated, monitored emergency account exists and is excluded from

the approver group it would otherwise gate. Implement: Create/confirm a cloud-only break-glass Global Administrator with a stored, sealed credential and exclude it from `Change Approvers – Apps – Prod`.

2.5 An admin-role design (least privilege)

Description: A written design of which admins need which permissions and which slice of objects. Why: Custom roles and scope tags are only as good as the least-privilege design behind them. Verify: A documented map of admin personas → permissions → scope; run `RBACGOV-Discovery.ps1` to inventory current roles and assignments as a baseline. Implement: Draft the persona-to-role map; define `App Manager – Region A – Prod` and `Policy Reader – All – Audit` from it.

2.6 A scope-tag taxonomy

Description: A consistent set of scope tags (for example by region, business unit, or platform). Why: Scope tags are the who-touches-what boundary; an inconsistent taxonomy defeats the purpose. Verify: Intune → Tenant administration → Roles → Scope tags; run `RBACGOV-Discovery.ps1`. Implement: Define the taxonomy; create `Region A – Devices – Prod` and apply it to that region's objects.

2.7 A list of high-risk resource types to gate

Description: The resource types whose changes should require a second approver (for example apps and scripts). Why: Multi-admin approval is configured per resource type; you must decide which to gate. Verify: Intune → Tenant administration → Multi admin approval → Access policies; run `RBACGOV-Validation.ps1`. Implement: Decide the gated types and create `App Deploy Approval – All – Prod` and `Script Deploy Approval – All – Prod`.

2.8 A change-control process for approvals

Description: An agreed process for who approves, the SLA to respond, and how rejections are communicated. Why: An approval gate with no process becomes a bottleneck and is bypassed. Verify: A documented approval SLA and approver roster; the approver group is populated. Implement: Define the SLA; populate `Change Approvers – Apps – Prod`.

2.9 Network access to Microsoft Graph / Intune endpoints

Description: Outbound HTTPS to Microsoft Graph and Intune service URLs for the PowerShell scripts. Why: The discovery, validation, and report scripts call Microsoft Graph. Verify: From the admin workstation, reach `https://graph.microsoft.com`. Implement: Allow Microsoft Graph / Intune URLs through proxy/firewall.

Checklist

- Intune licensing assigned
- Admin + approver Entra groups exist — `RBACGOV-Discovery.ps1`
- Operator holds Intune Administrator
- Break-glass Global Administrator exists and is excluded from the gate

- Least-privilege admin-role design documented
- Scope-tag taxonomy defined
- High-risk resource types to gate chosen
- Approval change-control process and SLA agreed
- Network allows Microsoft Graph / Intune endpoints

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** Phoenixtekk solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every Phoenixtekk solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The Phoenixtekk deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval)

Follow these steps in order, in the Microsoft Intune admin center (`https://intune.microsoft.com`).

> *Model: Design least-privilege roles and a scope-tag taxonomy, build admin/approver scope groups, create custom roles and assignments, apply scope tags to objects, then enable access policies (multi-admin approval) for high-risk resource types. Pilot one approval before broad enforce, and keep a break-glass admin so you never lock yourself out.*

All artifact names below are used verbatim; reuse them exactly.

Step 1 — Build the admin and approver scope groups

1. Entra → Groups → New group → Security → `Intune Admins – Region A – Prod` (the admins who will

hold the custom role). Add members. Create.

1. New group → Security → `Change Approvers – Apps – Prod` (the second-approver group). Add the

approvers. Create.

1. Optionally build both groups in one pass with `RBACGOV-Set-ScopeTargeting.ps1`.

Step 2 — Create the scope tag

1. Intune → Tenant administration → Roles → Scope tags → Create.
2. Name: `Region A – Devices – Prod`. Next.
3. Assignments → add the groups/objects this tag should be visible to as needed. Create.

Step 3 — Create the custom roles

1. Intune → Tenant administration → Roles → All roles → Create → Custom role.

2. Name: `App Manager – Region A – Prod`. Under Permissions, enable only the app-management

permissions this persona needs (for example Mobile apps: Read/Create/Update/Assign); leave device and policy permissions off. Next.

1. Scope (Tags): select `Region A – Devices – Prod`. Next → Create.

2. Create a second custom role `Policy Reader – All – Audit` with read-only permissions across the

workloads an auditor must view, no write permissions. Create.

Step 4 — Assign the custom role

1. Tenant administration → Roles → All roles → open `App Manager – Region A – Prod` → Assignments →

Assign.

1. Assignment name: `App Manager Assignment – Region A – Prod`.

2. Admin Groups (members): add `Intune Admins – Region A – Prod`.

3. Scope (Groups): add the managed group(s) this role may administer.

4. Scope (Tags): select `Region A – Devices – Prod`. Next → Create.

Step 5 — Apply the scope tag to objects

1. For each app/policy/profile this region owns, open the object → Properties → Scope tags → add

`Region A – Devices – Prod` → Save.

1. This makes the object visible/manageable only to admins whose assignment carries the same tag.

Run `RBACGOV-Discovery.ps1` to confirm tag coverage.

Step 6 — Enable multi-admin approval for high-risk apps

1. Intune → Tenant administration → Multi admin approval → Access policies → Create.

2. Name: `App Deploy Approval – All – Prod`.

3. Profile type: select the apps resource type (so app create/change/delete requires approval).

4. Approvers: select `Change Approvers – Apps – Prod`. Create.

Step 7 — Enable multi-admin approval for high-risk scripts

1. Multi admin approval → Access policies → Create.

2. Name: `Script Deploy Approval – All – Prod`.

3. Profile type: select the scripts resource type.

4. Approvers: select `Change Approvers – Apps – Prod`. Create.

Step 8 — Pilot one approval (the pilot gate)

1. As an admin in `Intune Admins – Region A – Prod`, make a small change to a gated resource (for

example edit a test app assignment). The change is held as a pending request.

1. As a different admin in `Change Approvers – Apps – Prod`, go to Multi admin approval → Received

requests, review, and Approve with a note.

1. Confirm the change applies only after approval, and that a rejection blocks it. Run

`RBACGOV-Validation.ps1` to confirm the policy is active and the role assignment is correct.

Step 9 — Validate and report

1. Run `RBACGOV-Validation.ps1` → confirms role assignments and approval policies match the design.
2. Run `RBACGOV-Report.ps1` → governance coverage and any over-privileged assignments.
3. Complete the Testing Guide before expanding.

Step 10 — Expand to production

1. Roll the scope-tag taxonomy out to the remaining regions/business units; create additional custom

roles and assignments following the same naming pattern.

1. Add access policies for any other high-risk resource types once the pilot approval is proven.
2. Re-validate each change with `RBACGOV-Validation.ps1` / `RBACGOV-Report.ps1`.

> *Rollback note: if an assignment or approval gate is too tight, loosen or remove it using your break-glass Global Administrator (excluded from the gate).* `RBACGOV-Remediation.ps1` *runs with* `-WhatIf` *by default so you can preview any tightening before it is applied.*

Order of operations

`Scope groups → Scope tag → Custom roles → Role assignment → Tag objects → App approval policy → Script approval policy → Pilot approval → Validate/report → Expand`

5. Testing Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval)

Run after the pilot approval is configured (Implementation Step 8) and after each governance change.

Test 1 — Custom roles exist and are scoped

Do: Intune → Tenant administration → Roles → All roles → confirm `App Manager – Region A – Prod` and `Policy Reader – All – Audit`; or run `RBACGOV-Validation.ps1`. Expected: Both custom roles exist with only their intended permissions and scope tags.

Test 2 — Role assignment is correct

Do: Open `App Manager – Region A – Prod` → Assignments → confirm `App Manager Assignment – Region A – Prod`. Expected: Members = `Intune Admins – Region A – Prod`; scope tag = `Region A – Devices – Prod`; scope groups as designed.

Test 3 — Scope tags limit visibility

Do: Sign in as a member of `Intune Admins – Region A – Prod`. Expected: Only objects carrying `Region A – Devices – Prod` are visible/manageable; objects without the tag are hidden.

Test 4 — Least privilege holds

Do: As the app-manager admin, attempt an action outside the role (for example create a device compliance policy). Expected: The action is blocked / not available — the role grants app permissions only.

Test 5 — Access policies exist and target the approver group

Do: Tenant administration → Multi admin approval → Access policies → confirm `App Deploy Approval – All – Prod` and `Script Deploy Approval – All – Prod`; or run `RBACGOV-Validation.ps1`. Expected: Both policies exist, target the correct resource types, and approver = `Change Approvers – Apps – Prod`.

Test 6 — A gated change requires approval

Do: As an app-manager admin, change a gated app; as a different approver, review under Received requests. Expected: The change is held pending until a second admin approves; it does not apply on its own.

Test 7 — Rejection blocks the change

Do: Submit another gated change and have the approver reject it with a note. Expected: The change is not applied; the requester sees the rejection.

Test 8 — Governance coverage report

Do: Run `RBACGOV-Report.ps1`. Expected: A clear report of roles, scope-tag coverage, assignments, approval-policy coverage, and any over-privileged assignments flagged.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass on the pilot scope; one approval and one rejection behave correctly
Broad pass	Tests 1–6, 8 pass across the expanded scope before declaring governance complete

6. Troubleshooting Guide — Intune Administrative Governance (RBAC & Multi-Admin Approval)

Use `RBACGOV-Discovery.ps1` and `RBACGOV-Validation.ps1` for current state, `RBACGOV-Report.ps1` for coverage/over-privilege, and `RBACGOV-Remediation.ps1` (`-WhatIf` by default) for safe fixes.

A. Admin can see/do too much or too little

Symptom	Cause	Remediation
Admin sees objects they should not	Object missing the scope tag, or assignment carries the wrong tag	Apply `Region A – Devices – Prod` to the object; confirm the assignment's scope tag with `RBACGOV-Validation.ps1`
Admin cannot see objects they should manage	Object not tagged, or admin in the wrong group	Tag the object; confirm membership of `Intune Admins – Region A – Prod`
Admin can perform actions outside their role	Over-broad custom role or a second, wider assignment	Run `RBACGOV-Report.ps1` to find over-privilege; tighten with `RBACGOV-Remediation.ps1`

B. Multi-admin approval not gating changes

Symptom	Cause	Remediation
Gated change applies without approval	Access policy missing or wrong resource type	Confirm `App Deploy Approval – All – Prod` / `Script Deploy Approval – All – Prod` exist and target the right type — `RBACGOV-Validation.ps1`
No approvers can act	Approver group empty or wrong group selected	Populate `Change Approvers – Apps – Prod`; confirm the policy targets it
Requester can self-approve	Requester is also in the approver group	Remove requesters from the approver group — separation of duties requires distinct people

C. Locked out (no one can make changes)

- Symptom: every admin is blocked by the approval gate or by an over-tight role.
- Remediation: sign in with the break-glass Global Administrator (excluded from the gate), then loosen or remove the offending access policy or assignment. This is why the break-glass account in requirement 2.4 is mandatory.

D. Pending approval requests stuck

- Approvers not responding within the SLA → escalate per the change-control process (requirement 2.8); confirm the approver group is populated and notifications reach approvers.

E. Scripts cannot read governance data

- The discovery/validation/report scripts need Microsoft Graph access and the right read scopes (for example `DeviceManagementRBAC.Read.All`, `DeviceManagementConfiguration.Read.All`). Confirm the operator consented to the requested scopes and can reach `https://graph.microsoft.com`.

F. Diagnostics quick reference

```
Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All'
(Invoke-MgGraphRequest -Method GET -Uri '/beta/deviceManagement/roleDefinitions').value | Select-Object di
(Invoke-MgGraphRequest -Method GET -Uri '/beta/deviceManagement/roleScopeTags').value | Select-Object disp
(Invoke-MgGraphRequest -Method GET -Uri '/beta/deviceManagement/operationApprovalPolicies').value | Select
```

G. When to escalate

If roles, scope tags, and access policies match the design (confirmed by `RBACGOV-Validation.ps1`), the approver group is populated, and the break-glass path works, but approvals still do not behave as expected, capture the `RBACGOV-Report.ps1` output and open a Microsoft support case.

7. Best Practices — Intune Administrative Governance (RBAC & Multi-Admin Approval)

- Least privilege by default. Start every admin from a narrow custom role (for example `App Manager – Region A – Prod`), not the full Intune Administrator role; grant only the permissions the persona actually uses.
- Always keep a break-glass admin. Maintain a cloud-only Global Administrator excluded from the custom roles and from the approver group, so a too-tight policy can never lock everyone out.
- Design the scope-tag taxonomy first. A consistent tag scheme (region/business unit/platform) is the who-touches-what boundary; apply tags to objects as you create them, not as an afterthought.

• Separate request from approval. Configure access policies so the admin who makes a high-risk change

is never the one who approves it — that is the whole point of multi-admin approval.

• Gate the changes that actually hurt. Start with the highest-blast-radius resource types (apps,

scripts) via `App Deploy Approval – All – Prod` and `Script Deploy Approval – All – Prod`; add more only as the process matures.

• Pilot one approval before broad enforce. Prove an approval and a rejection on a test change before

enforcing the gate across the fleet (the pilot gate).

• Set an approval SLA. An approval gate with no response-time commitment becomes a bottleneck; agree

who approves and how fast.

• Name everything consistently. Use `<Purpose> – <Scope> – <Stage>` for every role, tag, group,

assignment, and policy so the model is self-documenting and auditable.

• Review for over-privilege regularly. Run `RBACGOV-Report.ps1` on a schedule to catch assignments

that drifted broader than the design.

• Remediate safely. `RBACGOV-Remediation.ps1` defaults to `-WhatIf`; preview every tightening before

applying, and apply during a change window.

• Document the governance model. Record roles, tags, assignments, gated resource types, and the

approver roster for audits and repeatability.

8. Deliverables Summary — Intune Administrative Governance (RBAC & Multi-Admin Approval)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What Intune RBAC + multi-admin approval is, the model, benefits, limitations, and the why
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps (roles, scope tags, assignments, access policies)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation

Document	Purpose
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`RBACGOV-Discovery.ps1`	Inventories role definitions, scope tags, role assignments, and access policies (read-only)
`RBACGOV-Validation.ps1`	Validates role assignments and approval policies against the design (read-only)
`RBACGOV-Report.ps1`	Governance-coverage and over-privilege report (read-only)
`RBACGOV-Remediation.ps1`	Safely tightens over-privileged assignments; defaults to `-WhatIf`
`RBACGOV-Set-ScopeTargeting.ps1`	Builds the admin/approver scope groups

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the Phoenixtekk model: Assess → Configure → Pilot → Validate → Expand.

Outcome

Intune locked down to least privilege — custom roles and scope tags limiting who can touch what, and a required second-admin approval on high-risk changes — proven on a pilot before fleet-wide enforce.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

RBACGOV-Discovery.ps1

Purpose: Read-only inventory of the Intune administrative governance surface: custom and built-in role definitions, scope tags, role assignments (with their scope tags and members), and multi-admin approval access policies. Use it as a baseline before designing least-privilege roles and approval gates. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RBACGOV-Discovery.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS Console listing of role definitions (built-in vs custom), scope tags, role assignments, and access policies. Read-only — no changes are made. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All','DeviceManagementConfiguration.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Discovery.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS Console listing of role definitions (built-in vs custom), scope tags, role assignments, and access policies. Read-only — no changes are made. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All','DeviceManagementConfiguration.Read.All' -NoWelcome

RBACGOV-Remediation.ps1

Purpose: Tightens over-privileged role assignments by applying a target scope tag to assignments that currently have none (the most common over-privilege: an assignment with unbounded reach). Defaults to -WhatIf so it only previews changes; pass -Apply to actually write. HOW TO RUN Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\RBACGOV-Remediation.ps1 -ScopeTagId 5 Apply changes: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Remediation.ps1 -ScopeTagId 5 -Apply Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.ReadWrite.All (write only happens with -Apply; preview needs read only). EXPECTED RESULTS In preview: a WHATIF line per assignment that would be tightened. With -Apply: each no-scope-tag assignment receives $ScopeTagId. Always run RBACGOV-Validation.ps1 afterward. Keep a break-glass admin excluded from the gate so you never lock yourself out. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Defaults to -WhatIf (no changes). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ScopeTagId, # the roleScopeTag id to apply to unbounded assignments [switch]$Apply # omit for -WhatIf preview (default)) $whatIf = -not $Apply if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop $scopes = if ($Apply) { @('DeviceManagementRBAC.ReadWrite.All') } else { @('DeviceManagementRBAC.Read.All') } Connect-MgGraph -Scopes $scopes -NoWelcome

How to run: Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\RBACGOV-Remediation.ps1 -ScopeTagId 5 Apply changes: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Remediation.ps1 -ScopeTagId 5 -Apply Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.ReadWrite.All (write only happens with -Apply; preview needs read only). EXPECTED RESULTS In preview: a WHATIF line per assignment that would be tightened. With -Apply: each no-scope-tag assignment receives $ScopeTagId. Always run RBACGOV-Validation.ps1 afterward. Keep a break-glass admin excluded from the gate so you never lock yourself out. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Defaults to -WhatIf (no changes). #> [CmdletBinding()] param([Parameter(Mandatory)][string]$ScopeTagId, # the roleScopeTag id to apply to unbounded assignments [switch]$Apply # omit for -WhatIf preview (default)) $whatIf = -not $Apply if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop $scopes = if ($Apply) { @('DeviceManagementRBAC.ReadWrite.All') } else { @('DeviceManagementRBAC.Read.All') } Connect-MgGraph -Scopes $scopes -NoWelcome

RBACGOV-Report.ps1

Purpose: Read-only report summarizing governance coverage and flagging over-privileged assignments: how many roles are custom vs built-in, scope-tag coverage, which assignments lack a scope tag, which assignments use a broad built-in role, and which high-risk resource types are NOT covered by an access policy (multi-admin approval). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RBACGOV-Report.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All. EXPECTED RESULTS A coverage summary plus an OVER-PRIVILEGE section listing assignments with no scope tag or a broad built-in role, and gated-resource gaps. Read-only — no changes are made. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Report.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All. EXPECTED RESULTS A coverage summary plus an OVER-PRIVILEGE section listing assignments with no scope tag or a broad built-in role, and gated-resource gaps. Read-only — no changes are made. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param() if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All' -NoWelcome

RBACGOV-Set-ScopeTargeting.ps1

Purpose: Creates (idempotently) the Entra security groups used by this solution: the admin group that custom-role assignments target, and the approver group that multi-admin approval access policies target. Run once during setup; safe to re-run (skips groups that already exist). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RBACGOV-Set-ScopeTargeting.ps1 Requires the Microsoft Graph SDK and consent to Group.ReadWrite.All. Edit $AdminGroup / $ApproverGroup below if you use different names. EXPECTED RESULTS The groups 'Intune Admins – Region A – Prod' and 'Change Approvers – Apps – Prod' exist (created if missing). Add members in Entra, then use them on the role assignment and access policies. This script creates groups only; it does not assign roles. .NOTES Phoenixtekk Workplace

Marketplace — Intune Solutions. Creates Entra groups (write). #> [CmdletBinding()] param([string]$AdminGroup = 'Intune Admins – Region A – Prod', [string]$ApproverGroup = 'Change Approvers – Apps – Prod') if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Set-ScopeTargeting.ps1 Requires the Microsoft Graph SDK and consent to Group.ReadWrite.All. Edit $AdminGroup / $ApproverGroup below if you use different names. EXPECTED RESULTS The groups 'Intune Admins – Region A – Prod' and 'Change Approvers – Apps – Prod' exist (created if missing). Add members in Entra, then use them on the role assignment and access policies. This script creates groups only; it does not assign roles. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Creates Entra groups (write). #> [CmdletBinding()] param([string]$AdminGroup = 'Intune Admins – Region A – Prod', [string]$ApproverGroup = 'Change Approvers – Apps – Prod') if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All' -NoWelcome

RBACGOV-Validation.ps1

Purpose: Read-only check that the expected custom roles, role assignments (with scope tags), and access policies (multi-admin approval) exist and target the right approver group. Confirms the governance design is actually in force. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RBACGOV-Validation.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All. Edit the $expected* variables below to match your design before running. EXPECTED RESULTS PASS/FAIL lines for each expected role, assignment, and access policy. Read-only — no changes. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string[]]$ExpectedRoles = @('App Manager – Region A – Prod','Policy Reader – All – Audit'), [string[]]$ExpectedPolicies = @('App Deploy Approval – All – Prod','Script Deploy Approval – All – Prod'), [string] $ApproverGroup = 'Change Approvers – Apps – Prod') if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All' -NoWelcome

How to run: powershell -ExecutionPolicy Bypass -File .\RBACGOV-Validation.ps1 Requires the Microsoft Graph SDK and consent to DeviceManagementRBAC.Read.All. Edit the $expected* variables below to match your design before running. EXPECTED RESULTS PASS/FAIL lines for each expected role, assignment, and access policy. Read-only — no changes. .NOTES Phoenixtekk Workplace Marketplace — Intune Solutions. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string[]]$ExpectedRoles = @('App Manager – Region A – Prod','Policy Reader – All – Audit'), [string[]]$ExpectedPolicies = @('App Deploy Approval – All – Prod','Script Deploy Approval – All – Prod'), [string] $ApproverGroup = 'Change Approvers – Apps – Prod') if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'DeviceManagementRBAC.Read.All' -NoWelcome